INTRODUCTION TO DEEP LEARNING

Transformers

Presented by **Massa Baali**

Carnegie Mellon University

Outline

PART 1	**Understand the limitation:** Why Transformers? How does a Transformer process a sequence?
PART 2	**Understand the architecture:** Attention, positional information, and inference.
PART 3	**Understand how it learns:** Pretraining, SFT, instruction tuning, and post-training.
PART 4	**Understand how it scales:** Mixture of Experts and sparse computation.
PART 5	**Understand how it expands beyond text:** Tokenization, images, audio, and multimodal models.

Context changes meaning

“bank”

I went to the **bank** to apply for a loan.

We had a picnic on the river **bank**.

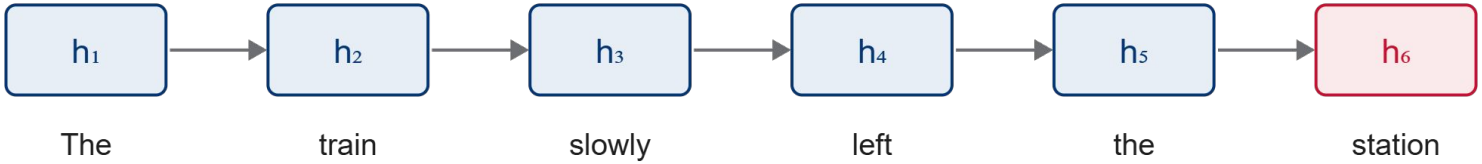
“it”

The animal didn’t cross the street because **it** was too **tired**.

The animal didn’t cross the street because **it** was too **wide**.

> One fixed vector per word can’t capture this. We need embeddings that depend on the **context**.

Pre-Transformers: RNNs read one word at a time



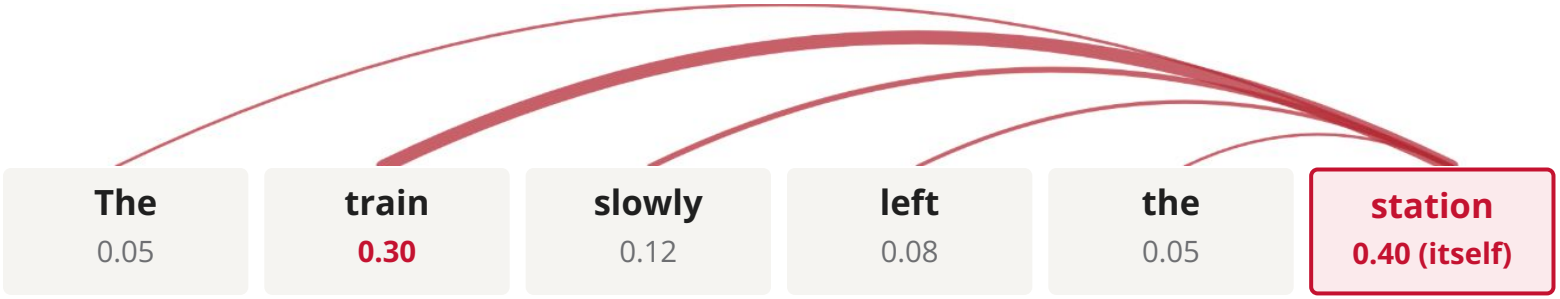
Problem 1: Sequential

RNNs by design must process sequences one step at a time. h_6 needs h_5, which needs h_4. **No parallelism.**

Problem 2: Bottleneck

Everything about the past must be compressed into **one fixed-size vector**. Important details can be lost or become difficult to retrieve as the sequence grows.

From embeddings to contextual embeddings




```
station' = 0.05·The + 0.30·train + 0.12·slowly + … + 0.40·station
```

Weighted Average
Output vector is a dynamic linear combination of all input embeddings.

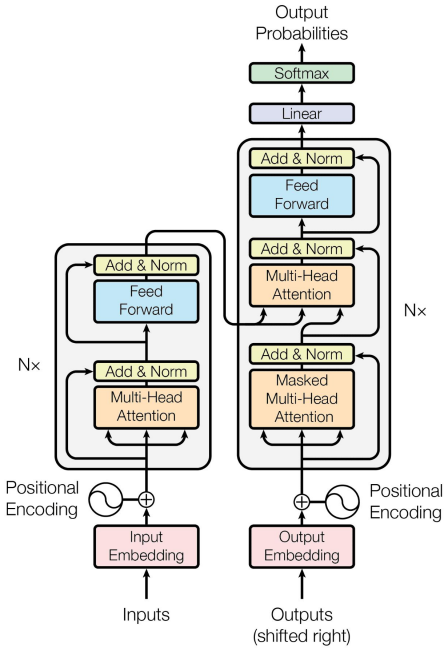
Solves Bottleneck
Direct 1-step connections eliminate information loss across long sequences.

Full Parallelism
All token positions computed simultaneously, unlike sequential RNN steps.

Outline

PART 1	**Understand the limitation:** Why Transformers? How does a Transformer process a sequence?
PART 2	**Understand the architecture:** Attention, positional information, and inference.
PART 3	**Understand how it learns:** Pretraining, SFT, instruction tuning, and post-training.
PART 4	**Understand how it scales:** Mixture of Experts and sparse computation.
PART 5	**Understand how it expands beyond text:** Tokenization, images, audio, and multimodal models.

Transformer Architecture Overview



Attention
Tokens exchange context dynamically across sequence lengths.

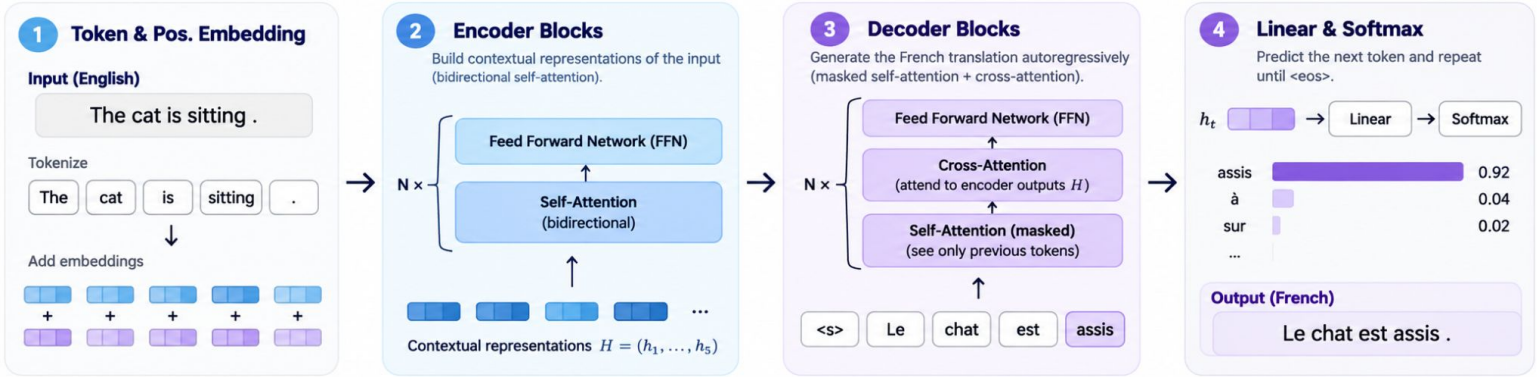
Feed-forward
Each token is processed on its own by the same small MLP.

Residual + norm
Stabilizes deep gradient flow for dozen-layer models.

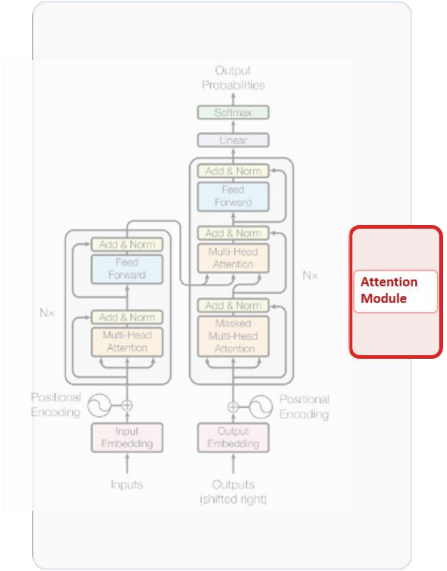
Positional Encodings
Injects absolute/relative order into parallel operations.

Original Transformer: N = 6 layers, d = 512, 8 attention heads, FFN width 2,048.

End-to-End Pipeline: English to French



Attention



WHY WAS IT ADDED BY DESIGN?

Enables tokens to dynamically exchange contextual information regardless of their distance in the sequence, bypassing sequential RNN recurrence bottlenecks.

What it Enforces

Pairwise token interaction with O(1) path length across sequence positions.

Multi-Head Benefit

Attends to information from different representation subspaces simultaneously.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) V$$

Attention Weights

1 · Relatedness
Score each pair with a dot product: w_i · w_j is large when two embeddings point the same way.

2 · Normalize
A softmax turns the scores into weights that are positive and sum to 1.

3 · Average
The new vector is the weighted sum: w_i′ = Σ_j s_ij w_j.

```
s_ij = exp(w_i · w_j) / Σ_k exp(w_i · w_k)
```

But one vector is doing three jobs
Asking (“what am I looking for?”), being matched (“what do I contain?”), and being passed along (“what do I contribute?”). Let’s give each job its own learned projection.

Queries, keys and values

Each token gets three learned projections of its embedding x_i:

Query

`q_i = x_i W_Q`

What am I looking for?

Library: the question you walk in with

Key

`k_j = x_j W_K`

What do I contain?

Library: the label on each book's spine

Value

`v_j = x_j W_V`

What do I hand over if chosen?

Library: the contents of the book

> Match your **query** against every **key** → weights → take that mix of **values**.

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) V$$

Q, K, V
n × d_k each

→

$QK^T / \sqrt{d_k}$
n × n scores

→

softmax by row
each row sums to 1

→

× V
n × d_k output

Fully parallel

Every row is computed at once with two matrix multiplies, which GPUs do extremely well.

Quadratic in length

Every token scores every token: the n × n matrix makes cost and memory grow as n^2.

Example: three tokens, d_k = 2

token	q	k	v
the	[1,0]	[1,0]	[1,0]
cat	[0,1]	[0,2]	[0,1]
sat	[1,1]	[1,1]	[1,1]

Toy numbers chosen by hand; in a real model q, k, v come from xW_Q, xW_K, xW_V.

ROW FOR "SAT"

```
q·k = [1, 2, 2]
÷ √2 → [0.71, 1.41, 1.41]
softmax → [0.20, 0.40, 0.40]
z = 0.20·v_the + 0.40·v_cat + 0.40·v_sat
z = [0.60, 0.80]
```

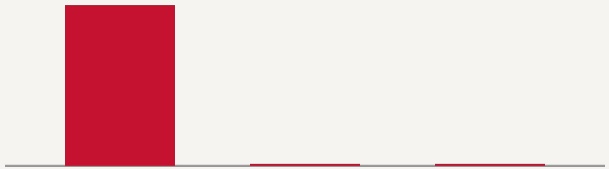
> "sat" ends up 40% "cat," 40% itself and 20% "the": its new vector now carries information about its subject.

Why divide by √d_k?

If the entries of q and k are roughly N(0, 1), then q · k has variance d_k: a typical score is about **±8** when d_k = 64.

Without scaling (d_k = 64)

`scores ≈ [8, 0, −8]`



softmax ≈ [1.00, 0.00, 0.00]: **one-hot, so gradients ≈ 0**

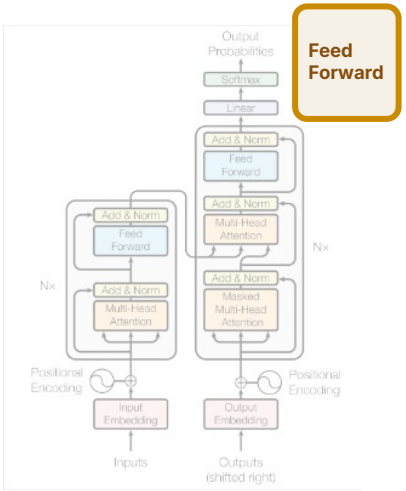
With scaling (÷ √64 = 8)

`scores ≈ [1, 0, −1]`



softmax = [0.67, 0.24, 0.09]: **smooth, so gradients flow**

Position-Wise Feed-Forward Block



WHY WAS IT ADDED BY DESIGN?

Attention only mixes linear combinations of tokens. The FFN adds necessary non-linear feature transformation and capacity to each token position individually.

Per-Token Processing

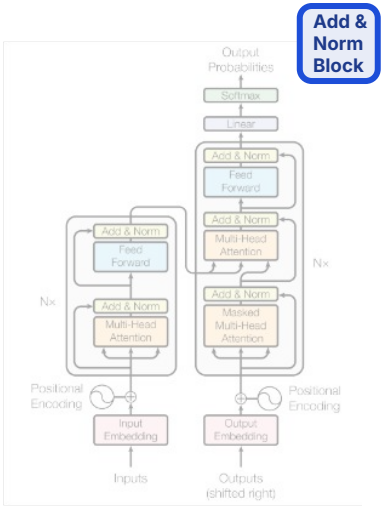
Applied identically to each position without cross-token interaction.

Dimensional Expansion

Projects d_model (512) to inner d_ff (2048) and back to d_model.

$$FFN(x) = \max(0, x W_1 + b_1) W_2 + b_2$$

Residual Connections & Layer Normalization



WHY WAS IT ADDED BY DESIGN?

Prevents vanishing/exploding gradients in deep networks, allowing information to pass directly through residual shortcuts without degradation.

Residual Path

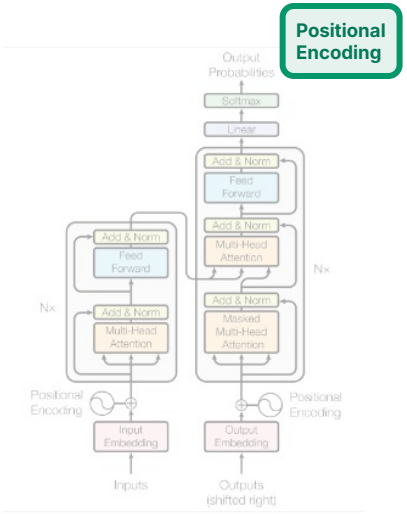
Preserves identity signal across blocks, enabling stacking of dozens of layers like LEGO.

Layer Normalization

Stabilizes activation distributions per token, independent of batch size.

```
Output = LayerNorm( x + SubLayer(x) )
```

Positional Encodings



WHY WAS IT ADDED BY DESIGN?

Attention operations are permutation-invariant (order-agnostic). Positional encodings explicitly inject sequence position information directly into the input embeddings.

No Recurrence

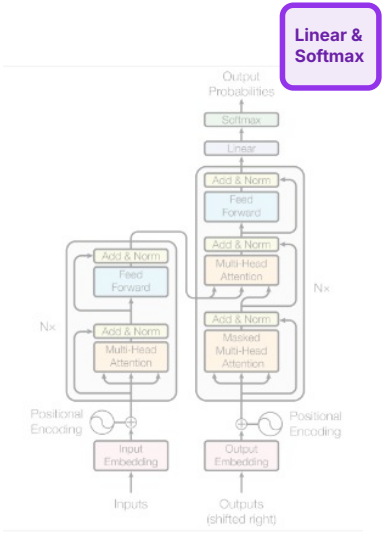
Preserves massive parallel execution while maintaining word order awareness.

Sinusoidal Functions

Allows model to easily learn relative positions via trigonometric linear relations.

```
PE(pos, 2i) = sin(pos / 10000^(2i/d))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d))
```

Linear Projection & Softmax Output



WHY WAS IT ADDED BY DESIGN?

Maps high-dimensional hidden representations back to the target vocabulary space to produce predicted probability distributions for token generation.

Linear Projection

Projects vector of size d_model to logits vector of size vocab_size.

Softmax Normalization

Converts unnormalized logits into normalized target probabilities summing to 1.

```
P(y_t | y_<t) = softmax( h_t W_vocab )
```

The causal mask: no peeking at the future

	The	cat	sat	on	mat
The	0	−∞	−∞	−∞	−∞
cat	0	0	−∞	−∞	−∞
sat	0	0	0	−∞	−∞
on	0	0	0	0	−∞
mat	0	0	0	0	0

Rows: queries · Columns: keys

Add a mask M to the scores before the softmax:

$$A = \text{softmax}(QK^T/\sqrt{d_k} + M)$$

M = 0 where j ≤ i, and **−∞** where j > i

exp(−∞) = 0, so future tokens get exactly zero weight

One forward pass trains every position of the sequence at once

POLL 1

Why do we divide by √d_k?

A. To make attention cheaper to compute

B. To keep scores from growing with d_k, so the softmax doesn't saturate

C. To make each row of weights sum to 1

D. To encode the position of each token

POLL 1 · ANSWER

Why do we divide by √d_k?

A. To make attention cheaper to compute

B. To keep scores from growing with d_k, so the softmax doesn't saturate

C. To make each row of weights sum to 1

D. To encode the position of each token

Attention has no sense of order

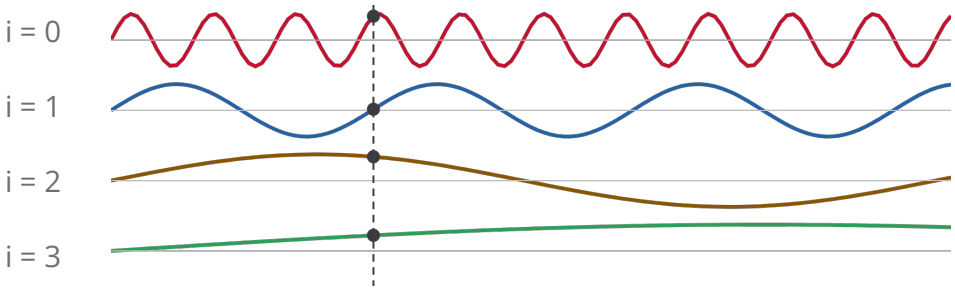
INPUT	dog	bites	man	→	z_dog	z_bites	z_man
SHUFFLED	man	bites	dog	→	z_man	z_bites	z_dog

Shuffle the inputs and the outputs shuffle the same way, but each vector is **identical**. Self-attention is **permutation-equivariant**: “dog bites man” and “man bites dog” look the same to it.

> **Fix:** give each position its own signal, e.g. x_i′ = x_i + p_i

Sinusoidal positional encoding

```
PE(pos, 2i) = sin( pos / 10000^(2i/d) )
PE(pos, 2i+1) = cos( pos / 10000^(2i/d) )
```



Position 0 → 64 (left to right), d = 16. Dashed line: position 20 reads one value from every wave.

Clock hands at different speeds. Fast dimensions tell neighbors apart; slow ones tell distant positions apart.

Wavelengths run from 2π to 10,000 · 2π, so every position gets a **unique code**.

Shifts are rotations. PE(pos + k) is a fixed linear map of PE(pos), so relative offsets are easy to learn.

Learned positions: absolute and relative

Learned absolute

BERT, GPT-2, ViT

A trainable vector per index, p_0 … p_(L−1), added to the token embedding.

+ Simple; the model learns whatever position code helps.

− Nothing exists past L (512 in BERT, 1,024 in GPT-2), and “position 7” and “position 8” start as unrelated vectors.

Learned relative

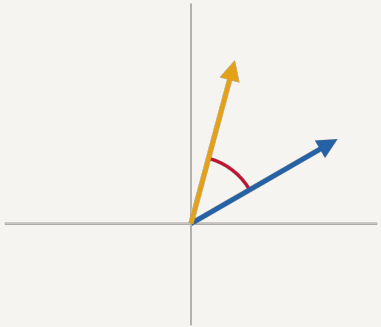
Shaw et al., 2018 · T5

The attention score gets a learned term that depends on the offset j − i, not on the absolute indices.

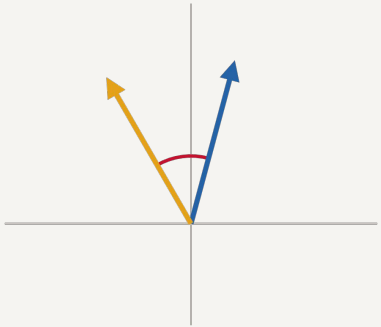
```
Shaw: e_ij = q_i · (k_j + a_(j−i)) / √d
T5: e_ij = q_i · k_j + b_bucket(j−i)
```

+ “Two tokens to my left” means the same everywhere in the sequence.

RoPE: encode position by rotating queries and keys



Word at m = 2, word at n = 5



Both shifted by 3: m = 5, n = 8

Rotate q at position m by m·θ and k at position n by n·θ (here θ = 15°; each pair of dimensions gets its own θ_i = 10000^(−2i/d)).

Shift both words by the same amount and both rotate equally: **the angle between them, and so their score, is unchanged.**

Applied to Q and K (not V) in every layer. Used in Llama, Mistral, Qwen and most open LLMs.

Why rotation gives relative position

$$q_m = R(m\theta)\, q \text{ and } k_n = R(n\theta)\, k$$

$$q_m^T k_n = q^T R(m\theta)^T R(n\theta) k = q^T R((n-m)\theta) k$$

→ **the score depends only on the offset n − m**

Why it works

Rotations compose: undoing a rotation by mθ and then rotating by nθ is one rotation by (n − m)θ.

Analogy

Two clock hands set some time apart: turn the whole clock face and the angle between them stays the same.

Longer contexts

Rescale the frequencies to stretch a trained model to longer inputs: position interpolation, NTK-aware scaling, YaRN.

Positional encodings compared

Method	Where it enters	Absolute or relative	Longer than training?	Used in
Sinusoidal	Added to input embeddings	Absolute (offsets recoverable)	In principle; poorly in practice	Original Transformer, Whisper encoder
Learned absolute	Added to input embeddings	Absolute	No: fixed table size	BERT, GPT-2, ViT
Learned relative bias	Added to attention scores	Relative (bucketed)	Partly: far offsets share a bucket	T5
ALiBi	Distance penalty on scores	Relative	Yes: train short, test long	BLOOM, MPT
RoPE	Rotates Q and K in every layer	Relative, via rotation	With frequency rescaling	Llama, Mistral, Qwen

The trend: from **adding** position once at the input to building **relative** position into every attention score.

POLL 2

RoPE injects position information by...

A. Adding a learned vector to each token embedding

B. Rotating query and key vectors by angles proportional to position

C. Adding a learned bias to the attention scores

D. Concatenating the position index to the input

POLL 2 · ANSWER

RoPE injects position information by…

A. Adding a learned vector to each token embedding

B. Rotating query and key vectors by angles proportional to position

C. Adding a learned bias to the attention scores

D. Concatenating the position index to the input

A is learned absolute (BERT, GPT-2); C is T5's relative bias. Nobody uses D.

Inference: generation one token at a time

Step	The model sees	Next token
1	The cat	sat
2	The cat sat	on
3	The cat sat on	the
4	The cat sat on the	mat

Training is parallel

The causal mask lets one forward pass score every position of a known sequence (teacher forcing).

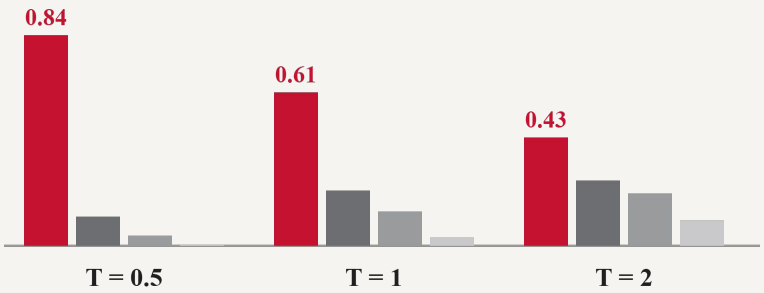
Generation is sequential

Each new token needs a full forward pass and depends on the token chosen before it. Stop at an end-of-sequence token or a length limit.

Choosing the next token

Temperature reshapes the distribution

`p = softmax(z / T)`



Bars: **mat**, sofa, roof, moon with logits 2, 1, 0.5, −1. P(mat) falls from 0.84 to 0.61 to 0.43 as T rises.

Greedy

Always take the most likely token.

Deterministic, but prone to dull, repetitive text.

Top-k

Sample only among the k most likely tokens; the long tail of junk is cut off.

Top-p (nucleus)

Sample from the smallest set whose probabilities add up to p (e.g., 0.9). The set shrinks when the model is confident.

KV cache: don’t recompute the past

STEP 5 OF GENERATION, ONE LAYER

	The	cat	sat	on	the
Q	–	–	–	–	q_5
K	k_1	k_2	k_3	k_4	k_5
V	v_1	v_2	v_3	v_4	v_5

Gray: read from the cache · **Red:** computed this step

Why it works

With a causal mask, earlier tokens never look at later ones, so their keys and values never change.

What we save

Each step computes q, k, v for one new token and appends k, v to the cache, instead of recomputing all t tokens in every layer.

```
bytes per token = 2 × layers × kv_heads × head_dim × bytes
```

POLL 3

With a KV cache, what do we store for previous tokens?

A Their queries, keys and values

B Only their keys and values

C Their attention weights

D Their output probabilities

POLL 3 · ANSWER

With a KV cache, what do we store for previous tokens?

A Their queries, keys and values

B Only their keys and values

C Their attention weights

D Their output probabilities

Outline

PART 1	**Understand the limitation:** Why Transformers? How does a Transformer process a sequence?
PART 2	**Understand the architecture:** Attention, positional information, and inference.
PART 3	**Understand how it learns:** Pretraining, SFT, instruction tuning, and post-training.
PART 4	**Understand how it scales:** Mixture of Experts and sparse computation.
PART 5	**Understand how it expands beyond text:** Tokenization, images, audio, and multimodal models.

One architecture. Three learning objectives.

What a Transformer can see determines what it can learn to predict.

BERT · Encoder-only

Attention: bidirectional, sees both sides

Trained to: fill in masked tokens

Example: The [MASK] left → train

GPT · Decoder-only

Attention: causal, sees only the past

Trained to: predict the next token

Example: The train left → station

T5 · Encoder–decoder

Attention: bidirectional encoder + causal decoder

Trained to: map one sequence to another

Example: translation

Same attention building blocks; **different masks, information flow, and targets.**

Pretraining: the data creates its own supervision

INPUT	A	robot	must	obey	the
TARGET	robot	must	obey	the	orders

$$L(\theta) = -\Sigma_t \log p_\theta(x_t \mid x_1, \ldots, x_{t-1})$$

No labels needed

Any text is a training example, so we can train on trillions of tokens from the web, books and code.

n predictions per pass

With the causal mask, a sequence of n tokens yields n next-token predictions in one forward pass.

Pretraining: the data creates its own supervision

Connect the output softmax to the Transformer weights.

STEP 1

Predict

Transformer hidden states → vocabulary logits → softmax probabilities.

→

STEP 2

Measure error

Cross-entropy penalizes low probability assigned to the correct token.

→

STEP 3

Update

Backpropagate through attention, FFNs, embeddings and output head.

```
L_pretrain = − Σ_t log p_θ( x_t | x_<t )
```

Different pretraining losses

BERT · masked LM

```
The [MASK] left the [MASK]
→ train, station
```

Mask 15% of tokens and predict them from both sides. Great encoders, but not generators.

GPT · next token

```
The train left the
→ station
```

Predict each token from the ones before it. Generation comes for free.

T5 · span corruption

```
The <X> the station
→ <X> train left
```

Generate missing spans using encoder–decoder attention.

Today's LLMs are almost all **decoder-only next-token predictors**: one objective, one model, and any task can be phrased as text.

The problem: prediction is not instruction following

A base model is optimized to continue text, not necessarily to help a user.

> **Prompt:** Translate “Good morning” into French.

Base model (pre-trained only)

Translate “Good evening” into Spanish.

Desired response

Bonjour.

How do we train the same decoder-only Transformer to respond to requests?

Post-training: same architecture

Post-training is a stage; SFT and preference optimization are methods within it.

STAGE 1

Pretraining

Learn statistical structure from large-scale text.

→

STAGE 2

SFT / instruction tuning

Learn from demonstrations of useful responses.

→

STAGE 3

Preferences

Learn to favor preferred answers.

LoRA is an efficient way to update parameters during these stages.

SFT: next-token prediction, now on the answer

The same next-token loss as pre-training, but on curated (prompt, response) pairs, and only on the response.



Gray: prompt tokens, no loss · **Red outline:** response tokens, loss computed

$$L = -\Sigma_{(t \text{ in response})} \log p_\theta(x_t \mid x_{<t})$$

Data

High-quality demonstrations: answers, multi-turn dialogues, code, step-by-step solutions, refusals.

What it changes

Mostly behavior and format: answer instead of continue, follow the chat format, know when to stop.

Instruction tuning: SFT across many tasks

The objective stays similar; the diversity of instructions changes.

Translate

INPUT

“Good morning”

OUTPUT

“Bonjour”

Summarize

INPUT

Long article

OUTPUT

Concise summary

Explain

INPUT

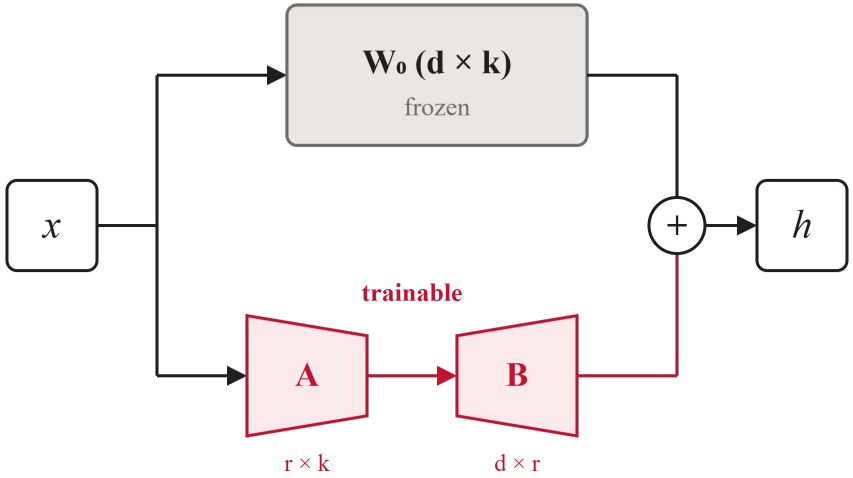
“What is attention?”

OUTPUT

Simple analogy

One Transformer learns a general pattern: interpret the instruction and produce an answer.

LoRA: Low-Rank Adaptation of Large Language Models



$$h = W_0 x + (\alpha / r) \cdot BAx$$

W_0 stays **frozen**; only **A and B** are trained

Rank $r \ll d, k$ (e.g., $r = 8$)

Hypothesis: the change needed to adapt a model has low intrinsic rank

LoRA: adapt attention without updating every weight

Parameter-efficient fine-tuning changes how we update weights, not the loss.

Full fine-tuning

Update the entire pretrained matrix W.

High optimizer memory and many trainable parameters.

Low-rank adaptation

Freeze W. Train small A and B.

Apply to attention projections such as W_Q and W_V.

```
W_new = W_frozen + B · A (rank r ≪ min(d_in, d_out))
```

Beyond demonstrations: which response is better?

SFT learns what a good answer looks like; preferences compare alternatives.

> **PROMPT:** Explain self-attention to a beginner.

Response A

"Compute pairwise compatibility scores and normalize..."

Response B · preferred

"Imagine each word looking at the others..."

Preference data: (prompt, **chosen answer**, rejected answer)

DPO: learn from chosen vs. rejected answers

Unlike SFT, preference optimization compares two completions.

> **PROMPT:** Explain self-attention to a beginner.

SFT objective

Increase likelihood of demonstrated target tokens.

One target response.

DPO objective

Increase relative preference for chosen over rejected, relative to a reference policy.

A pair of responses.

DPO uses sequence log-probabilities, a reference model, and a preference loss.

One Transformer. A complete learning journey.

Keep the architecture, data, objective, and trainable parameters conceptually separate.

Stage	Learning signal	What changes
Pretraining	Self-supervised tokens	Usually all weights
SFT	Demonstrated responses	All weights or adapters
Instruction tuning	Diverse task instructions	SFT specialization
LoRA	Same task loss as fine-tuning	Low-rank adapters
DPO	Chosen / rejected responses	Policy weights or adapters

Pretraining learns to predict. Post-training learns to respond. LoRA makes adaptation efficient.

POLL 4

In LoRA, why is B initialized to zero?

A To save memory during training

B So BA = 0 and training starts exactly from the pre-trained model

C To make the update full-rank

D Because B is frozen during training

POLL 4 · ANSWER

In LoRA, why is B initialized to zero?

A. To save memory during training

B. So BA = 0 and training starts exactly from the pre-trained model

C. To make the update full-rank

D. Because B is frozen during training

Outline

PART 1	**Understand the limitation:** Why Transformers? How does a Transformer process a sequence?
PART 2	**Understand the architecture:** Attention, positional information, and inference.
PART 3	**Understand how it learns:** Pretraining, SFT, instruction tuning, and post-training.
PART 4	**Understand how it scales:** Mixture of Experts and sparse computation.
PART 5	**Understand how it expands beyond text:** Tokenization, images, audio, and multimodal models.

From Learning to Scaling: Mixture of Experts

We know how Transformers learn. **Must every parameter work for every token?**

Dense Transformer

One FFN processes every token

Shared FFN

All FFN weights active

MoE Transformer

Router selects a few expert FFNs

E1 **E2** E3 E4

Router

BIG IDEA

Increase total model capacity while activating only selected experts per token.

ANALOGY: A HOSPITAL

Many specialists on staff, but an intake nurse (the router) sends each patient to two of them, not to every doctor in the building.

The mixture-of-experts layer




```
y = Σ g_i(x) · E_i(x), i in top-k
```

The FFN in each block is replaced by E expert FFNs plus a router; attention stays dense and shared. Here **E = 8 and k = 2**, as in Mixtral.

Where Does MoE Change the Transformer?

Recall: attention mixes tokens; the feed-forward network transforms each token.



MoE replaces the dense FFN with a router + multiple expert FFNs.

The attention mechanism can remain unchanged.

Example: A Router Selects Two Experts

Four experts, top-2 selection for one token.

Expert 1	Expert 2	Expert 3	Expert 4
48%	31%	15%	6%
SELECTED	SELECTED	SKIPPED	SKIPPED

Output = weighted combination of selected expert outputs

Router scores depend on the token representation; exact normalization varies by MoE design.

Why Not Use MoE Everywhere?

Sparse activation reduces expert computation, but introduces **new systems challenges**.

Load imbalance

Too many tokens may choose the same expert.

Communication

Experts may live on different accelerators.

Memory

All expert weights still need storage.

MoE scales **parameter capacity**, not necessarily latency or memory efficiency in every setting.

POLL 5

An MoE layer has 8 experts with top-2 routing. Which part of the block do the experts replace?

- **A** The attention layer
- **B** The feed-forward network
- **C** The token embeddings
- **D** The whole transformer block

POLL 5 · ANSWER

An MoE layer has 8 experts with top-2 routing. Which part of the block do the experts replace?

A The attention layer

B The feed-forward network

C The token embeddings

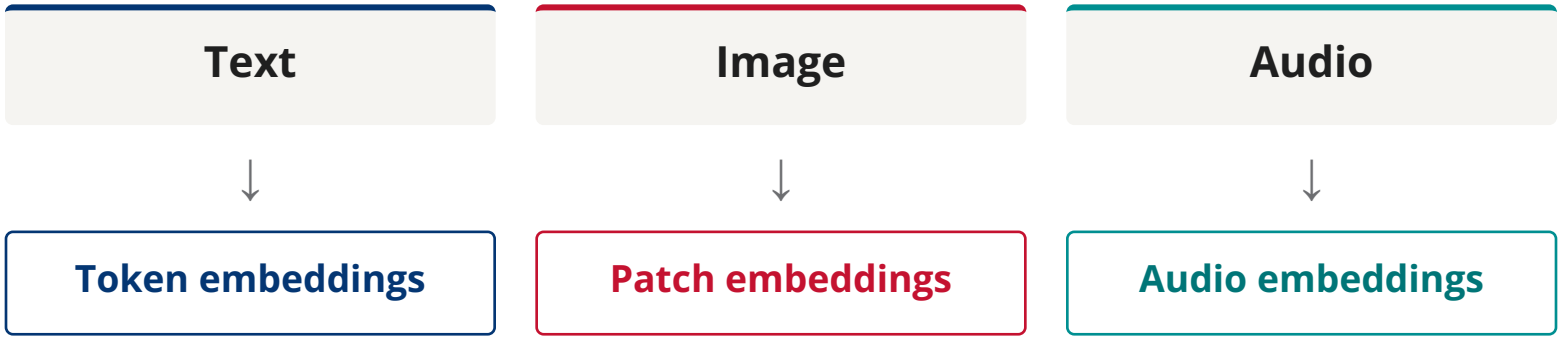
D The whole transformer block

Outline

PART 1	**Understand the limitation:** Why Transformers? How does a Transformer process a sequence?
PART 2	**Understand the architecture:** Attention, positional information, and inference.
PART 3	**Understand how it learns:** Pretraining, SFT, instruction tuning, and post-training.
PART 4	**Understand how it scales:** Mixture of Experts and sparse computation.
PART 5	**Understand how it expands beyond text:** Tokenization, images, audio, and multimodal models.

Beyond Text: The Transformer Sees Vectors

What changes when the input is an **image** or a **sound** instead of a sentence?



Different front ends produce sequences of vectors for attention.

Text Tokenization

Characters

`t·r·a·n·s·f·o·r·m·e·r·s`

\+ Tiny vocabulary; nothing is ever unknown

– Very long sequences, and attention costs n^2

Words

`transformers`

\+ Short sequences

– Huge vocabulary; unseen words become [UNK]

Subwords

`transform·ers`

\+ Common words stay whole; rare words split into known pieces

\+ The standard: BPE, WordPiece, Unigram

Everything is a token

Text	subword pieces
Images	16 × 16 pixel patches
Audio	~20 ms frames, or discrete codec codes

Once the input is a sequence of vectors, the transformer *doesn't care where it came from.*

Vision Transformer: an image is worth 16×16 words

1 · PATCHIFY

Cut the image into 16 × 16 squares

→

2 · FLATTEN

A sequence of patches, read row by row

→

3 · EMBED

Linear projection of each patch, plus a learned position embedding; prepend a [CLS] token

→

4 · ENCODE

A standard transformer encoder: every patch attends to every patch

→

5 · CLASSIFY

An MLP head on the [CLS] output → “cat”

No convolutions after the patch projection: **the same encoder we built for text, with patches as tokens.**

Audio: from waveform to tokens

16 kHz audio is 16,000 numbers per second: a 30-second clip is **480,000 samples**. Far too long for n^2 attention, so we compress first.

1 · Spectrogram frames

Log-Mel features every 10 ms, then convolution layers downsample in time.

e.g., Whisper

2 · Learned conv features

A CNN reads the raw waveform and emits one vector per ~20 ms.

e.g., wav2vec 2.0, HuBERT

3 · Discrete codec tokens

A neural codec turns audio into integer codebook IDs, just like text token IDs.

e.g., SoundStream, EnCodec → AudioLM, VALL-E

Audio's version of a 16 × 16 patch is a short frame of about **20–25 ms**.

Whisper: an encoder–decoder for speech

Audio
30-second window, 16 kHz

→

Log-Mel
80 channels × 3,000 frames (10 ms hop)

→

2 conv layers
stride 2 → 1,500 frame tokens

→

Encoder
Transformer with sinusoidal positions

→

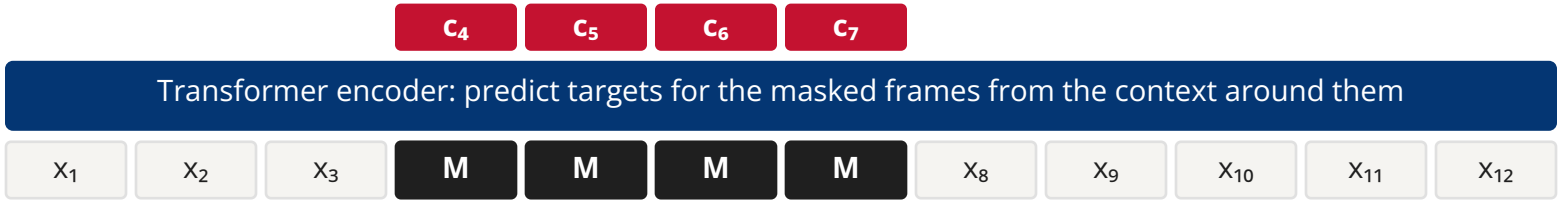
Decoder
Writes text tokens, cross-attending to the encoder

DECODER OUTPUT: TASK TOKENS, THEN THE TRANSCRIPT

```
<|startoftranscript|><|en|><|transcribe|><|notimestamps|> The cat sat on the mat.
```

Special tokens tell **one model** which task to do: transcribe, translate to English, identify the language, add timestamps.

Self-supervised speech: masked prediction



CNN features from raw audio, one per 20 ms; a span of frames is masked (M)

wav2vec 2.0

Pick the true quantized target for each masked frame among distractors (contrastive). 10 minutes of labels + 53k hours unlabeled: 4.8 / 8.2 WER on LibriSpeech clean / other.

HuBERT

Predict k-means cluster IDs of the masked frames, BERT-style. Iterate: cluster MFCCs first, then the model's own features.

DELULU

HuBERT-style masked prediction, but the pseudo-labels come from a speaker model (ReDimNet), so the features keep who is speaking, not just what is said.

How Images and Text Meet

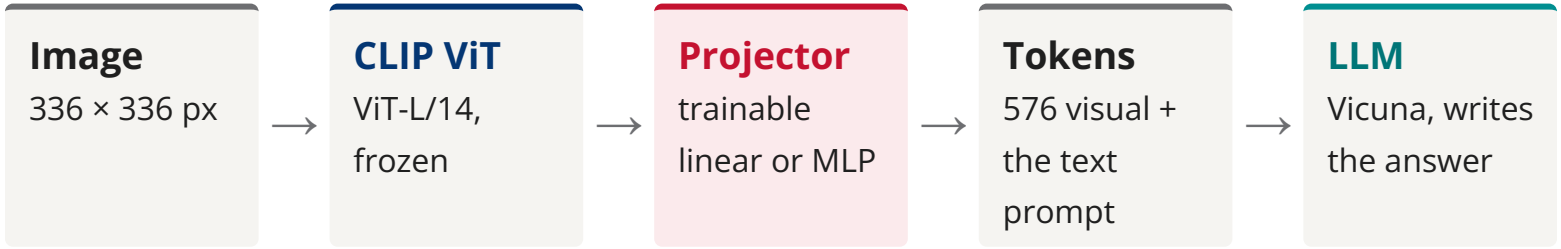
A vision-language model connects visual representations to language modeling.



Text prompt + visual features → grounded response

One common design; other multimodal architectures fuse modalities differently.

LLaVA: a multimodal LLM in two stages



Stage 1 · Feature alignment
Train only the projector on 595K image–caption pairs. Vision encoder and LLM both frozen.

Stage 2 · Visual instruction tuning
Train projector + LLM on 158K instruction-following samples. Vision encoder stays frozen.

576 = $(336 / 14)^2$ patches, as in LLaVA-1.5. The original LLaVA used a linear projector at 224 px; LLaVA-1.5 moved to an MLP at 336 px.

Key takeaways

1 Attention is learned weighted averaging

Every token builds its representation from every other token, all positions in parallel.

2 Order comes from positional encodings

Attention alone is order-blind. RoPE builds relative position into every score.

3 LLMs = pre-training + post-training

Next-token prediction at scale gives knowledge; SFT and preferences give behavior; LoRA makes adapting cheap.

4 Everything is a token

Patches, audio frames and codec codes share one architecture; MoE grows parameters, not per-token compute.

INTRODUCTION TO DEEP LEARNING

Thank you :-)

Questions? · **Massa Baali**